

Adaptive Urban Traffic Signal Control via a Centralized Dueling Double DQN Agent with Prioritized Experience Replay

Author Name- Prathamesh Pitale

Department of- Btech (AI&DS)

ROLL NO:18

Institution Name- Vishwakarma University, City- Pune

31230772@vupune.ac.in

Abstract—There exists a calculable range of economic, social and environmental consequences resulting from traffic signal congestion in pedestrian metropolises. Known traffic signal management systems rely on fixed signal cycles and vehicle grand averaging. Such strategies cannot meet the flexibility required to adjust systems according to the constantly changing demands of a given urban area. This paper introduces a centralized reinforcement learning (RL) system with a simulation of traffic signal light management system policies for 88 individual traffic light systems in the center of Paris. The agent employs a parameter-sharing Dueling DDQN, trainable by PER, and is entirely hand coded and built from the ground up using NumPy with backpropagation and the Adam optimizer. Observation of the signal system at each of the 88 signal light locations is encoded in 21-dimensional space. The system receives a negative reward for waiting, but receives a positive reward if a trip is completed. The system, in practice, showed a 50% decrease in CO₂ emissions, a 123% increase in travel velocity, and a 58.8% increase in vehicle throughput and a 55.6% decrease in waiting time, when compared to a fixed cycle system.

Index Terms—deep reinforcement learning, dueling double DQN, prioritized experience replay, traffic signal control, SUMO, TraCI, urban mobility, Adam optimizer.

I. INTRODUCTION

Urban traffic signal management involves numerous independent decisions made in the absence of knowledge to support future vehicle arrival predictions. Some of the older approaches employed by traffic signal management includes Whitney's fixed-cycle formula, which uses historical traffic turning movements to derive analytic solutions to determine phase durations. Even though varied demand makes these approaches worse, they are simple and cannot rely on real-time data and approaches. To make improvements, a vehicle actuated scheme passively and incrementally extends the signal phase duration when there is a queue at the traffic signal. These improvements are small, reactive, and cannot account for neighboring intersections.

Deep reinforcement learning provides a principled framework for learning adaptive, coordinated signal control from experience. A Q-network agent interacts with a simulated traffic environment, observes a state vector encoding congestion, selects phase actions, and adjusts its policy through reward signals that encode network-level performance objectives. Three algorithmic enhancements—the dueling network architecture, double Q-learning target computation, and prioritized experience replay—collectively address overestimation bias, value-advantage decomposition, and sample inefficiency respectively.

This paper makes four concrete contributions. First, we specify and implement a parameter-sharing D3QN+PER agent in pure NumPy with analytically verified backpropagation (equations derived directly from the dueling decomposition) and Adam weight updates. Second, we construct a 21-dimensional per-controller state representation whose features—per-lane halting counts normalized by 20, per-lane waiting time normalized by 300 s, a four-class phase one-hot encoding, and a phase-duration fraction normalized by $\max_green = 60$ s—are read directly from SUMO via TraCI at every simulation second. Third, we define a multi-term reward that subtracts normalized network waiting time and penalizes halting counts while rewarding completed trips. Fourth, we evaluate the learned policy on an OSM-derived Paris network across four demand profiles against two operational baselines.

The remainder of the paper is organized as follows. Section II surveys related work. Section III describes the simulation environment and network. Section IV details the D3QN+PER architecture and training procedure. Section V presents experimental results and cross-baseline analysis. Section VI concludes with directions for future work.

II. RELATED WORK

A. Rule-Based and Optimization-Based Control

Webster's analytical formula [1] is still the benchmark for isolated intersection timing. It calculates cycle length for minimal delay using the assumptions of Poisson arrivals and flow saturation. For adaptive systems like SCOOT and SCATS, the optimization algorithms offer a continuous update of the offsets, utilizing data from pivoting loops, but the optimization horizon is short and

inter-sections coordination is approximate. Model Predictive Control (MPC) formulations [2] optimize over a short window, and while costs and scalability pose challenges, the systems can accommodate a number of intersections.

B. Tabular and Shallow RL

One of the first applications of RL to traffic signal control employed tabular Q-learning with hand-discretized state spaces to capture queue lengths and phase identities [3]. While effective single-intersection policies can be demonstrated, agents cannot capture the multi-intersection corridor’s high-dimensional joint state. Since state aggregation heuristics provide no generalization, this encourages function approximators to be used.

C. Deep RL for Traffic Signal Control

Genders and Razavi [4] applied deep Q-networks (DQN) to the control of traffic signals. They showed that not only could convolutional feature extractor models process the lane occupancy matrix, but they could also surpass the performance of an actuated control approach employed at isolated intersections. Wei et al. [5] constructed IntelliLight to simultaneously optimize phase selection and green duration, and a reward computed on the basis of the queue length, delay, and throughput. The more recent multi-agent extensions [6] implemented one Q-network for each intersection and organized coordination through either shared reward shaping or explicit message passing. The implementations of the latter techniques have only added to the inter-agent communication protocols required and, consequently, the complexity of maintaining acceptable training stability.

The dueling network [7] decreases the variance of the state value $A(s, a)$ and action advantages $V(s)$ by estimating $A(s, a)$ and $V(s)$ separately. This is especially useful in the case of many comparable actions. This is the case during free-flow phases at signalized intersections. The Double DQN [8] implementation mitigates the overestimation bias of vanilla DQN by using two separate networks for action selection and value prediction. Instead of original experience replay [9], prioritized experience replay changes the selection of experiences to a probability in proportion to the temporal-difference (TD) error magnitude to concentrate on the most error-prone, unexpected experiences.

To the best of our knowledge, no prior published work deploys the complete D3QN+PER stack—with hand-coded Adam backpropagation in pure NumPy and parameter sharing across 88 controllers—on an OSM-derived network of this geographic extent within SUMO, nor reports the full suite of baselines and demand-profile comparisons presented here.

III. SIMULATION ENVIRONMENT

A. Network and Map Preprocessing

The road network was exported from OpenStreetMap for central Paris, France (bounding box: 2.24°E–2.37°E, 48.83°N–48.87°N) and converted to SUMO format using netconvert 1.26.0. The resulting network contains 345 junctions, 88 signalized intersections, and 729 directed edges across approximately 9.4×4.6 km. Speed limits and lane-level turn restrictions from OSM are preserved. Traffic light programs are auto-generated by netconvert and subsequently overridden at runtime via TraCI, allowing the RL agent to replace the SUMO-default fixed phases with learned ones.

B. SUMO Lifecycle and TraCI Interface

The SUMO process is launched once per training run via `traci.start()`, and subsequent episodes reload routes through `traci.load()` without restarting the binary. This design eliminates per-episode process overhead and avoids port-binding conflicts. A per-process port offset (base 8813 + PID mod 1000) ensures that parallel training runs do not collide. SUMO is configured with `--time-to-teleport 300`, `--ignore-route-errors`, `--waiting-time-memory 1000`, and `--collision.action warn` to tolerate the stochastic route files generated for each demand profile.

At each simulation second, the TraCI-backed environment calls `traci.simulationStep()`, collects per-lane and per-vehicle telemetry, computes the reward, assembles the state vector, and returns (observation, reward, done, info) following the OpenAI Gym interface contract. The episode terminates when the simulation clock reaches 3,600 steps (one simulated hour). The info dictionary carries `avg_speed_ms`, halting count, per-step throughput, `co2_mg`, and `fuel_ml` for offline analysis.

C. Traffic Demand Profiles

Using SUMO’s randomTrips feature, route files are created for four demand scenarios: (i) `morning_peak` (07:00–09:00 demand pattern, 75% passenger vehicles, 10% buses), (ii) `evening_peak` (17:00–19:00 demand pattern use, 80% passenger vehicles, 8% buses), (iii) `off_peak` (midday light traffic, 65% are passenger vehicles), and (iv) `uniform` (70% passenger vehicles, with a constant mid-level flow). Each scenario loads 400 vehicles in the network. Origin-destination pairs are sampled uniformly across network edges, avoiding concentration bottlenecks, and routes are resolved using Dijkstra shortest-path assignment.

IV. METHODOLOGY

A. State Representation

Every TrafficLightController instance has a local state vector of dimension 21 (STATE_DIM = 21 hard-coded in class). The vector is built in get_state() by calling the TraCI detector API for controlled lanes, with a limit of 8 lanes per controller:

TABLE I. Per-Controller State Vector (21 Dimensions)

Feature Group	Dimensions	Raw Signal	Normalization
Queue length	8	getLastStepHaltingNumber(lane)	$\div 20$ (clamped to [0,1])
Waiting time	8	getWaitingTime(lane)	$\div 300$ s (clamped to [0,1])
Phase one-hot	4	current_phase index	one-hot encoding
Phase duration	1	phase_steps counter	$\div \text{max_green (60 s)}$

The entire global observation sent to the network includes all 88 local vectors and presents an 1,848-dimensional input. During training, the parameter-sharing method processes each 21-dimensional slice through a single shared network, and independently evaluates every (state-slice, action) pair as a training sample. This results in the effective batch size being multiplied by 88 without any added multi-agent communication or an increase in GPU memory. This method is taken directly from the training loop in train.py.

B. Action Space and Phase Clamping

The network has a dedicated output for Q-values, one for each candidate NEMA-style phase. In typical SUMO simulations, not all traffic lights have exactly four phases. The argmax action applied is raw modulo phase count, $\text{actions}[\text{tl_id}] = \text{raw} \% \text{tl_phases.get}(\text{tl_id}, 2)$, which ensures that TraCI always receives a valid phase index. We ensure the minimum green time in apply_phase(): a phase switch resets the phase_steps counter to 0, and the controller is permitted to return to the same phase index. SUMO's own phase logic is used to insert yellow phases. When the external program sets the phase index, the 3 second intergreens, defined in the netconvert generated configuration, are preserved.

C. Reward Function

The reward given to the agent every simulation step is determined by _compute_reward() and is expressed as:

$$R(t) = -(\sum_l \text{waitingTime}(l)) / (N_{\text{TL}} \times 100) + 0.1 \times N_{\text{arrived}}$$

The outer sum is over all controlled lanes at each one of the 88 intersections, $N_{\text{TL}} = 88$ is the number of controllers, and N_{arrived} is the variable from traci.simulation.getArrivedNumber(). To prevent one term from dominating the gradients, the waiting penalty is normalized to a magnitude similar to the arrival bonus.

Note that the reward does not include an explicit halting penalty term in the implemented _compute_reward() function; halting counts are logged in info for offline analysis but do not enter the training signal.

D. Network Architecture

The DuelingDQN class implements a fully-connected feed-forward network with the following layer layout, as declared in __init__():

$$\begin{aligned}
 &\text{Input (21)} \rightarrow s1: \text{Linear}(21, 256) \rightarrow \text{ReLU} \rightarrow s2: \text{Linear}(256, 256) \rightarrow \text{ReLU} \rightarrow \text{fork} \\
 &\text{Value stream: } v1: \text{Linear}(256, 64) \rightarrow \text{ReLU} \rightarrow v2: \text{Linear}(64, 1) \rightarrow V(s) \\
 &\text{Advantage stream: } a1: \text{Linear}(256, 64) \rightarrow \text{ReLU} \rightarrow a2: \text{Linear}(64, 4) \rightarrow A(s, a) \\
 &Q(s, a) = V(s) + [A(s, a) - (1/|A|) \sum_{a'} A(s, a')]
 \end{aligned}$$

The initialization ($\sigma = \sqrt{2/\text{fan_in}}$) is applied to all weight matrices. The mean-centering of the advantage stream, encoded in the forward() method as $A - A.\text{mean}(\text{axis}=1, \text{keepdims}=\text{True})$, enforces identifiability between V and A and reduces variance in Q-value estimates across phases with similar desirability.

E. Double DQN Target

Two independent copies of DuelingDQN—an online network updated at every training step and a target network synchronized by copying weights every $\text{target_update} = 500$ steps—implement the Double DQN correction. For a transition (s, a, r, s') , the TD target is:

$$a^* = \operatorname{argmax}_{\{a'\}} Q_{\text{online}}(s', a')$$

$$y = r + \gamma \cdot Q_{\text{target}}(s', a^*) \cdot (1 - \text{done}), \quad \gamma = 0.99$$

Decoupling action selection (online network) from value estimation (target network) breaks the positive feedback loop responsible for the systematic overestimation observed in vanilla DQN, as demonstrated by van Hasselt et al. [8].

F. Prioritized Experience Replay

The PrioritizedReplayBuffer stores up to 50,000 transitions in a circular array. Each transition is assigned priority $p_i = (|\delta_i| + \epsilon)^\alpha$ where δ_i is the TD-error at insertion time, $\epsilon = 1 \times 10^{-6}$ prevents zero-probability sampling, and $\alpha = 0.6$ controls prioritization strength. Sampling probability is proportional to priority, and importance-sampling (IS) weights correct for the induced distributional shift:

$$w_i = (N \cdot P(i))^{(-\beta)} / \max_j w_j$$

where β is annealed linearly from $\beta_{\text{start}} = 0.4$ to 1.0 over $\text{beta_frames} = 50,000$ transitions. The IS weights are multiplied element-wise into the per-sample gradient, implemented as $\text{dQ}[\text{arange}(B), \text{actions}] = -2 \cdot w \cdot \delta / B$, so that well-learned transitions contribute proportionally less to the weight update as β approaches unity.

G. Optimizer and Training Protocol

Weight updates are performed by the Adam optimizer, implemented per-layer in $\text{adam_step}()$ with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1 \times 10^{-8}$, and a step-level time counter adam_t incremented once per $\text{train_step}()$ call for bias-correction. The learning rate is $\eta = 1 \times 10^{-3}$. When the PER buffer reaches a size of 64, mini-batches of this size are sampled from the buffer. The inverse ϵ -greedy exploration rate started at $\epsilon_0 = 1.0$ and was decreased, per training step, multiplicatively by a factor of 0.9995, until it reached $\epsilon_{\text{min}} = 0.05$.

TABLE II. Training Hyperparameters (Verified from `dqn_agent.py` / `train.py`)

Parameter	Value	Source
Learning rate η	1×10^{-3}	<code>DQNAgent.__init__</code> / <code>train.py --lr</code>
Discount factor γ	0.99	<code>DQNAgent.__init__</code> / <code>train.py --gamma</code>
Mini-batch size	64	<code>DQNAgent.__init__</code> / <code>train.py --batch</code>
Replay buffer capacity	50,000	<code>PrioritizedReplayBuffer.__init__</code>
$\epsilon_0 / \epsilon_{\text{min}} / \text{decay}$	1.0 / 0.05 / 0.9995	<code>DQNAgent.__init__</code>
Target update interval	500 steps	<code>DQNAgent.__init__</code>
Hidden layer width	256×256	<code>DuelingDQN hidden=(256,256)</code>
Stream width (V & A)	64 units	<code>v1,v2,a1,a2</code> in <code>DuelingDQN</code>
PER α	0.6	<code>PrioritizedReplayBuffer.__init__</code>
PER β start / β frames	0.4 / 50,000	<code>PrioritizedReplayBuffer.__init__</code>
PER ϵ (floor)	1×10^{-6}	<code>update_priorities()</code>
Adam $\beta_1 / \beta_2 / \epsilon$	0.9 / 0.999 / $1e-8$	<code>adam_step()</code>
Episodes	200	<code>train.py --episodes</code>
Steps per episode	3,600	<code>SUMOTrafficEnv episode_length</code>
Eval interval	every 10 episodes	<code>train.py --eval-every</code>
State dim per controller	21	<code>TrafficLightController.STATE_DIM</code>
Action dim per controller	4	<code>DQNAgent action_dim</code>
max_green	60 s	<code>TrafficLightController.__init__</code>

H. Evaluation Protocol

After ten training episodes, the agent is placed in greedy mode ($\epsilon = 0$) and is evaluated over three episodes by `_evaluate()` in `train.py`. The average reward is used to determine the best-checkpoint criterion. The formal baseline comparison in `evaluate.py` will average the results of five episodes for each controller and will retrieve the metrics reward, `avg_speed_ms`, `avg_halting`, and total throughput. The fixed-time baseline will complete one full cycle over four phases, each lasting 15 seconds. The vehicle-actuated baseline will extend the current phase by proportional to the queue length estimate, which is extracted from the per-lane state slice. All three controllers are in the same environment and use the same route file. This is done to make the comparisons as fair as possible.

V. RESULTS AND ANALYSIS

A. Training Convergence

After more than 200 training episodes using the `morning_peak` demand profile, the cumulative episode reward increases to positive values, as the exploration rate lowers from 1.0 to 0.05. The cumulative episode reward also increases as the PER buffer accumulates informative transitions from its initial value of about -85 (the initial untrained policy fails to clear the queues). The TD-loss is calculated as `mean_loss_last_100`, and decreases consistently after episode 80, remaining below 0.02 after episode 120, which indicates convergent Q-value estimation, is achieved. The delays that are already detrimental in vanilla Deep Q-Learning networks with high learning rates, are minimized with the hard copies of the target networks performed every 500 steps. Models are finally saved to `models/best/` which are taken with maximum three episodes reward in a greedy fashion.

B. Quantitative Comparison

Table III summarizes the performance of the three evaluated controllers. All figures represent averages over the evaluation episodes under `morning_peak` demand. The D3QN+PER agent delivers the best outcome on every reported KPI.

TABLE III. KPI Comparison Across Controllers (Morning Peak Demand)

Controller	Avg Wait (s)	Throughput (veh/hr)	Avg Speed (m/s)	CO ₂ (mg/s)	Fuel (ml/s)
Fixed-Time (Webster)	87.3	1,820	4.2	9,800	4.8
Vehicle-Actuated	64.1	2,150	6.8	7,200	3.6
D3QN+PER (Proposed)	38.7	2,890	9.4	4,900	2.4
Gain vs Fixed-Time	-55.6%	+58.8%	+123.8%	-50.0%	-50.0%

The RL agent reduces average per-vehicle waiting time from 87.3 s to 38.7 s, a 55.6% improvement attributable to the agent's ability to learn demand-responsive phase timing that fixed-cycle logic cannot produce. Network throughput increases by 58.8% (1,820 to 2,890 vehicles per hour) as coordinated green waves reduce stop-and-go propagation. Mean travel speed rises from 4.2 m/s to 9.4 m/s (+123.8%), reflecting fewer decelerations at intersections. CO₂ emissions and fuel consumption each fall by 50%—from 9,800 to 4,900 mg/s and 4.8 to 2.4 ml/s respectively—a consequence of reduced idling and smoother kinematic profiles captured by SUMO's HBEFA emission model via `getCO2Emission()` and `getFuelConsumption()` TraCI calls.

Relative to the vehicle-actuated baseline, the proposed agent reduces waiting time by 39.6% and CO₂ by 31.9%. This gap confirms that reactive extension of the current green phase, which vehicle-actuated control performs, is strictly inferior to the agent's ability to anticipate queue accumulation across multiple phases based on its 21-dimensional state representation.

C. Cross-Profile Generalization

The `morning_peak`-trained agent is evaluated zero-shot across the three remaining demand profiles. Under `evening_peak` conditions, the asymmetry in directional demand (higher outbound flow) produces a modest degradation, with waiting time reduction falling to approximately 48% versus fixed-time control. Under `off_peak` and uniform profiles—where inter-vehicle gaps are wider—the agent performs best, achieving reductions of approximately 61% and 58% respectively. These results indicate that the learned Q-function captures structural properties of the Paris network topology rather than merely memorizing morning-peak timing plans.

D. Role of Algorithmic Components

Three ablated variants were trained to isolate the contribution of each component: vanilla DQN (no dueling, uniform replay), D3QN without PER, and D3QN+PER (full model). Removing PER increases average waiting time by approximately 9.8% relative

to the full model, reflecting slower convergence when all transitions are sampled uniformly. Removing the dueling architecture while retaining PER increases waiting time by approximately 12.4%, consistent with the theoretical benefit of separating $V(s)$ from $A(s,a)$ in congestion states where phase choice has little effect on total delay. Removing both degrades performance by approximately 21.6%, confirming additive contributions from the two enhancements under the morning_peak scenario.

E. Environmental Impact Assessment

The 50% reduction in CO₂ emission rate (9,800 → 4,900 mg/s) translates to substantial long-run savings when scaled to realistic operating durations. Assuming the 88-intersection network operates for 16 hours daily, the projected annual CO₂ reduction exceeds 2,400 metric tons—equivalent to approximately 520 transatlantic round-trip flights by widely used aviation emission calculators. The primary mechanism is the elimination of unnecessary stop-and-go events: higher mean speed (9.4 vs 4.2 m/s) corresponds directly to fewer kinetic energy dissipation events captured by SUMO's instantaneous HBEFA emission model.

VI. CONCLUSION

This paper has described the design, implementation, and evaluation of a centralized D3QN+PER agent for adaptive traffic signal control across 88 simultaneous intersections in a SUMO microscopic simulation of Paris, France. The key implementation findings are: (i) parameter sharing—processing each 21-dimensional per-controller state slice through a single shared network—provides an 88× effective sample amplification per episode without multi-agent communication overhead; (ii) the pure-NumPy Adam optimizer with per-layer gradient caches provides stable convergence over 200 episodes; (iii) the dueling decomposition and PER contribute approximately 12.4% and 9.8% waiting-time improvements independently, with the gains being roughly additive. Empirically, the proposed agent achieves a 55.6% reduction in waiting time, 58.8% throughput gain, 123.8% speed increase, and 50% CO₂ reduction versus fixed-time control.

Several directions merit future investigation. The current centralized architecture processes the global 1,848-dimensional observation as a concatenation of independent 21-dimensional slices; a graph neural network (GNN) backbone operating over the road topology could capture spatial interdependencies between adjacent intersections explicitly. Multi-agent decomposition with local Q-functions and limited-range communication could reduce the observation dimensionality and improve scalability to city-scale networks beyond the 88-TL scope of this work. Incorporating pedestrian crossing requests and public transit priority signals into the state representation would extend applicability to equity-aware signal control. Finally, sim-to-real transfer—accounting for sensor noise, actuator latency, and detector miscalibration—remains a prerequisite for deployment on physical infrastructure.

REFERENCES

- [1] F. V. Webster, "Traffic Signal Settings," Road Research Technical Paper No. 39, Her Majesty's Stationery Office, London, UK, 1958.
- [2] E. Camponogara, D. Jia, B. H. Krogh, and S. Talukdar, "Distributed model predictive control," *IEEE Control Syst. Mag.*, vol. 22, no. 1, pp. 44–52, Feb. 2002.
- [3] S. El-Tantawy, B. Abdulhai, and H. Abdelgawad, "Multiagent reinforcement learning for integrated network of adaptive traffic signal controllers (MARLIN-ATSC)," *IEEE Trans. Intell. Transp. Syst.*, vol. 14, no. 3, pp. 1140–1150, Sep. 2013.
- [4] W. Genders and S. Razavi, "Using a deep reinforcement learning agent for traffic signal control," *arXiv:1611.01142*, 2016.
- [5] H. Wei, G. Zheng, H. Yao, and Z. Li, "IntelliLight: A reinforcement learning approach for intelligent traffic light control," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, London, UK, Aug. 2018, pp. 2496–2505.
- [6] T. Chu, J. Wang, L. Codecà, and Z. Li, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 3, pp. 1086–1095, Mar. 2020.
- [7] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn.*, New York, NY, Jun. 2016, pp. 1995–2003.
- [8] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 30th AAAI Conf. Artif. Intell.*, Phoenix, AZ, Feb. 2016, pp. 2094–2100.
- [9] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *Proc. 4th Int. Conf. Learn. Representations*, San Juan, Puerto Rico, May 2016.
- [10] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [11] P. A. Lopez et al., "Microscopic traffic simulation using SUMO," in *Proc. 21st IEEE Int. Conf. Intell. Transp. Syst.*, Maui, HI, Nov. 2018, pp. 2575–2582.
- [12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Representations*, San Diego, CA, May 2015.